

Midterm Review

CS 230 – Distributed Systems

(<http://www.ics.uci.edu/~cs230>)

Nalini Venkatasubramanian
nalini@ics.uci.edu

Characterizing Distributed Systems

- Multiple Autonomous Computers
 - each consisting of CPU's, local memory, stable storage, I/O paths connecting to the environment
 - Geographically Distributed
- Interconnections
 - some I/O paths interconnect computers that talk to each other
- Shared State
 - No shared memory
 - systems cooperate to maintain shared state
 - maintaining global invariants requires correct and coordinated operation of multiple computers.

Classifying Distributed Systems

- Based on degree of synchrony
 - Synchronous
 - Asynchronous
- Based on communication medium
 - Message Passing
 - Shared Memory
- Fault model
 - Crash failures
 - Byzantine failures

Computation in distributed systems

- Asynchronous system
 - no assumptions about process execution speeds and message delivery delays
- Synchronous system
 - make assumptions about relative speeds of processes and delays associated with communication channels
 - constrains implementation of processes and communication
- Models of concurrency
 - Communicating processes
 - Functions, Logical clauses
 - Passive Objects
 - Active objects, Agents

Communication in Distributed Systems

- Provide support for entities to communicate among themselves
 - Centralized (traditional) OS's - local communication support
 - Distributed systems - communication across machine boundaries (WAN, LAN).
- 2 paradigms
 - Message Passing
 - Processes communicate by sharing messages
 - Distributed Shared Memory (DSM)
 - Communication through a virtual shared memory.

Fault Models in Distributed Systems

- Crash failures
 - A processor experiences a crash failure when it ceases to operate at some point without any warning. Failure may not be detectable by other processors.
 - Failstop - processor fails by halting; detectable by other processors.
- Byzantine failures
 - completely unconstrained failures
 - conservative, worst-case assumption for behavior of hardware and software
 - covers the possibility of intelligent (human) intrusion.

Client/Server Computing

- Client/server computing allocates application processing between the client and server processes.
- A typical application has three basic components:
 - Presentation logic
 - Application logic
 - Data management logic

Distributed Systems Middleware

- Middleware is the software between the application programs and the operating System and base networking
- Integration Fabric that knits together applications, devices, systems software, data
- Middleware provides a comprehensive set of higher-level distributed computing capabilities and a set of interfaces to access the capabilities of the system.

Virtual Time and Global States in Distributed Systems

Prof. Nalini Venkatasubramanian

Distributed Systems Middleware - Lecture 2

Includes slides modified from :

A. Kshemkalyani and M. Singhal (Book slides: Distributed Computing: Principles, Algorithms, and Systems)

Global Time & Global State of Distributed Systems

- Asynchronous distributed systems consist of several *processes* without common memory which communicate (solely) via *messages* with unpredictable transmission delays
- Global time & global state are hard to realize in distributed systems
 - Processes are distributed geographically
 - Rate of event occurrence can be high (unpredictable)
 - Event execution times can be small
- We can only *approximate* the global view
 - *Simulate synchronous* distributed system on given asynchronous systems
 - *Simulate a global time* – Logical Clocks
 - *Simulate a global state* – Global Snapshots

Simulating global time

- An accurate notion of global time is difficult to achieve in distributed systems.
 - We often derive “causality” from loosely synchronized clocks
- Clocks in a distributed system drift
 - Relative to each other
 - Relative to a real world clock
 - Determination of this real world clock itself may be an issue
 - Clock Skew versus Drift
 - Clock Skew = Relative Difference in clock values of two processes
 - Clock Drift = Relative Difference in clock frequencies (rates) of two processes
- Clock synchronization is needed to simulate global time
 - Correctness – consistency, fairness
- Physical Clocks vs. Logical clocks
 - Physical clocks - must not deviate from the real-time by more than a certain amount.

Physical Clocks

- How do we measure real time?
 - 17th century - Mechanical clocks based on astronomical measurements
 - Problem (1940) - Rotation of the earth varies (gets slower)
 - Mean solar second - average over many days
- 1948
 - counting transitions of a crystal (Cesium 133) used as atomic clock
 - TAI - International Atomic Time
 - 9192631779 transitions = 1 mean solar second in 1948
 - UTC (Universal Coordinated Time)
 - From time to time, we skip a solar second to stay in phase with the sun (30+ times since 1958)
 - UTC is broadcast by several sources (satellites...)

Cristian's (Time Server) Algorithm

- Uses a *time server* to synchronize clocks
 - Time server keeps the reference time (say UTC)
 - A client asks the time server for time, the server responds with its current time, and the client uses the received value T to set its clock
- But network round-trip time introduces errors...
 - Let **RTT = response-received-time – request-sent-time** (measurable at client),
 - If we know (a) $\min$ = minimum client-server one-way transmission time and (b) that the server timestamped the message at the last possible instant before sending it back
 - Then, the actual time could be between **$[T + \min, T + \text{RTT} - \min]$**

Berkeley UNIX algorithm

- One daemon without UTC
- Periodically, this daemon polls and asks all the machines for their time
- The machines respond.
- The daemon computes an average time and then broadcasts this average time.

Decentralized Averaging Algorithm

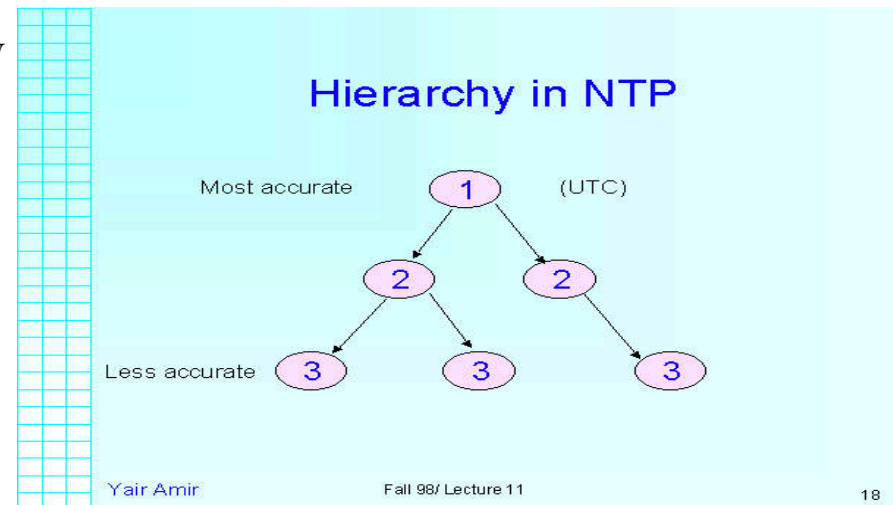
- Each machine has a daemon without UTC
- Periodically, at fixed agreed-upon times, each machine broadcasts its local time.
- Each of them calculates the average time by averaging all the received local times.

Clock Synchronization in DCE

- DCE's time model is actually in an interval
 - I.e. time in DCE is actually an interval
 - Comparing 2 times may yield 3 answers
 - $t1 < t2$
 - $t2 < t1$
 - not determined
 - Each machine is either a time server or a clerk
 - Periodically a clerk contacts all the time servers on its LAN
 - Based on their answers, it computes a new time and gradually converges to it.

Network Time Protocol (NTP)

- Most widely used physical clock synchronization protocol on the Internet
- 10-20 million NTP servers and clients in the Internet
- Claimed Accuracy (Varies)
 - milliseconds on WANs, submilliseconds on LANs
- Hierarchical tree of time servers.
 - The primary server at the root synchronizes with the UTC.
 - Secondary servers - backup to primary
 - Lowest
 - synchronization subnet with clients.



Logical Time

Causal Relations

- Distributed application results in a set of distributed events
 - Induces a partial order $\rightarrow$ causal precedence relation
- Knowledge of this causal precedence relation is useful in reasoning about and analyzing the properties of distributed computations
 - Liveness and fairness in mutual exclusion
 - Consistency in replicated databases
 - Distributed debugging, checkpointing

Event Ordering

- Lamport defined the “happens before” ($<$) relation
 - If a and b are events in the same process, and a occurs before b , then $a < b$.
 - If a is the event of a message being sent by one process and b is the event of the message being received by another process, then $a < b$.
 - If $X < Y$ and $Y < Z$ then $X < Z$.

If $a < b$ then $\text{time}(a) < \text{time}(b)$

Causal Ordering

- “Happens Before” also called causal ordering
- Possible to draw a causality relation between 2 events if
 - They happen in the same process
 - There is a chain of messages between them
- “Happens Before” notion is not straightforward in distributed systems
 - No guarantees of synchronized clocks
 - Communication latency

Implementing Logical Clocks

- Requires
 - Data structures local to every process to represent logical time and
 - a protocol to update the data structures to ensure the consistency condition.
- Each process P_i maintains data structures that allow it the following two capabilities:
 - A local logical clock, denoted by LC_i , that helps process P_i measure its own progress.
 - A logical global clock, denoted by GC_i , that is a representation of process P_i 's local view of the logical global time. Typically, LC_i is a part of GC_i
- The protocol ensures that a process's logical clock, and thus its view of the global time, is managed consistently.
 - The protocol consists of the following two rules:
 - R1: This rule governs how the local logical clock is updated by a process when it executes an event.
 - R2: This rule governs how a process updates its global logical clock to update its view of the global time and global progress.

Types of Logical Clocks

- Systems of logical clocks differ in their representation of logical time and also in the protocol to update the logical clocks.
- 3 kinds of logical clocks
 - Scalar
 - Vector
 - Matrix

Scalar Logical Clocks - Lamport

- Proposed by Lamport in 1978 as an attempt to totally order events in a distributed system.
- Time domain is the set of non-negative integers.
- The logical local clock of a process P_i and its local view of the global time are squashed into one integer variable C_i .
- Monotonically increasing counter
 - No relation with real clock
- Each process keeps its own logical clock used to timestamp events

Consistency with Scalar Clocks

- Local clocks must obey a simple protocol:
 - When executing an internal event or a send event at process P_i the clock C_i ticks
 - $C_i += d$ ($d > 0$)
 - When P_i sends a message m , it piggybacks a logical timestamp t which equals the time of the send event
 - When executing a receive event at P_i where a message with timestamp t is received, the clock is advanced
 - $C_i = \max(C_i, t) + d$ ($d > 0$)
- Results in a partial ordering of events.

Total Ordering

- Extending partial order to total order



- Global timestamps:
 - (T_a, P_a) where T_a is the local timestamp and P_a is the process id.
 - $(T_a, P_a) < (T_b, P_b)$ iff
 - $(T_a < T_b)$ or $((T_a = T_b) \text{ and } (P_a < P_b))$
 - Total order is consistent with partial order.

Vector Times

- The system of vector clocks was developed independently by Fidge, Mattern and Schmuck.
- In the system of vector clocks, the time domain is represented by a set of n -dimensional non-negative integer vectors.
- Each process has a clock C_i consisting of a vector of length n , where n is the total number of processes $vt[1..n]$, where $vt[j]$ is the local logical clock of P_j and describes the logical time progress at process P_j .
 - A process P_i ticks by incrementing its own component of its clock
 - $C_i[i] += 1$
 - The timestamp $C(e)$ of an event e is the clock value after ticking
 - Each message gets a piggybacked timestamp consisting of the vector of the local clock
 - The process gets some knowledge about the other process' time approximation
 - $C_i = \sup(C_i, t) :: \sup(u, v) = w : w[i] = \max(u[i], v[i]), \forall i$

Vector Clocks example

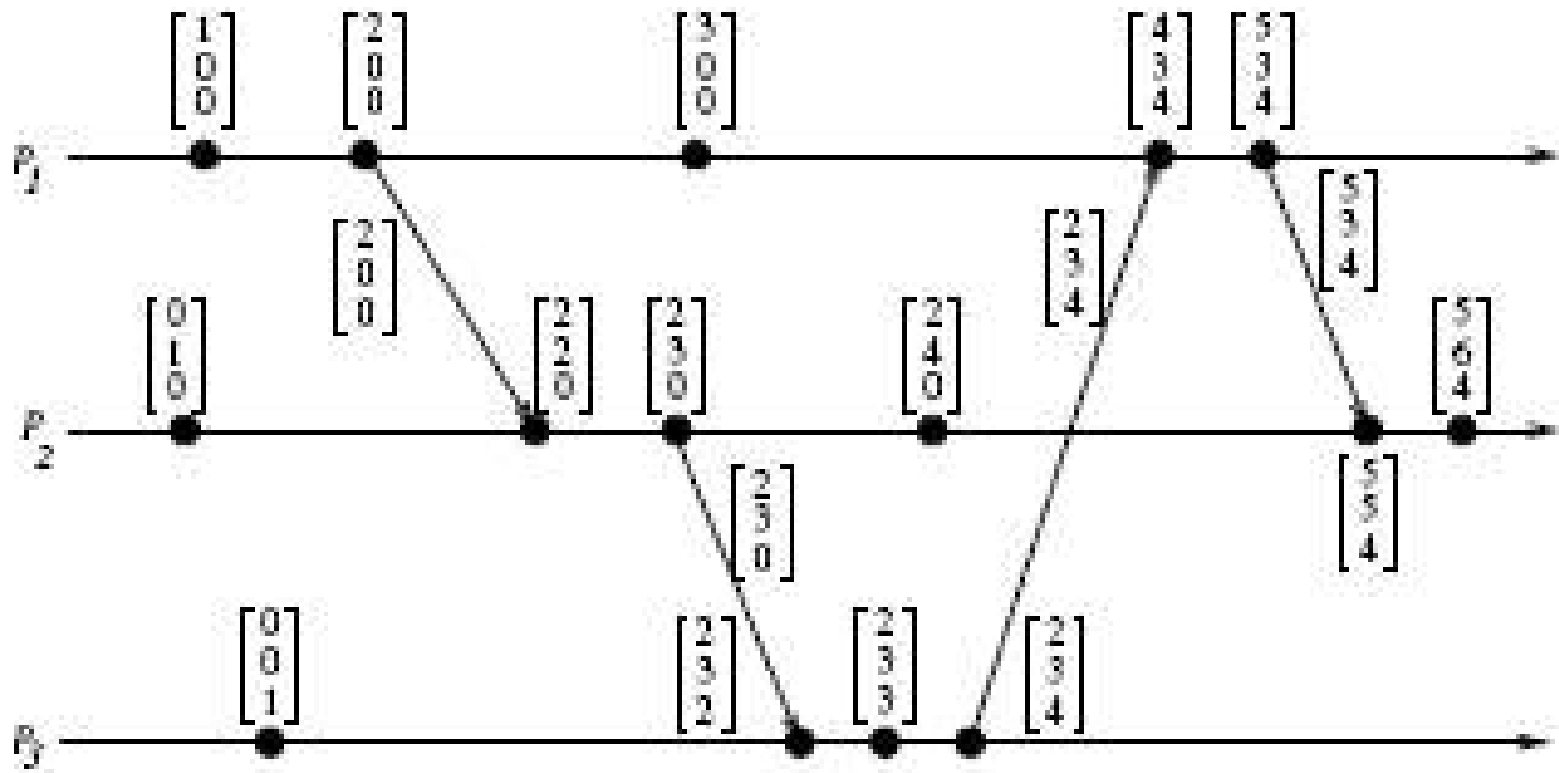


Figure 3.2: Evolution of vector time.

Matrix Time

- Vector time contains information about latest direct dependencies
 - What does P_i know about P_k
- Also contains info about latest direct dependencies of those dependencies
 - What does P_i know about what P_k knows about P_j
- Message and computation overheads are high
- Powerful and useful for applications like distributed garbage collection

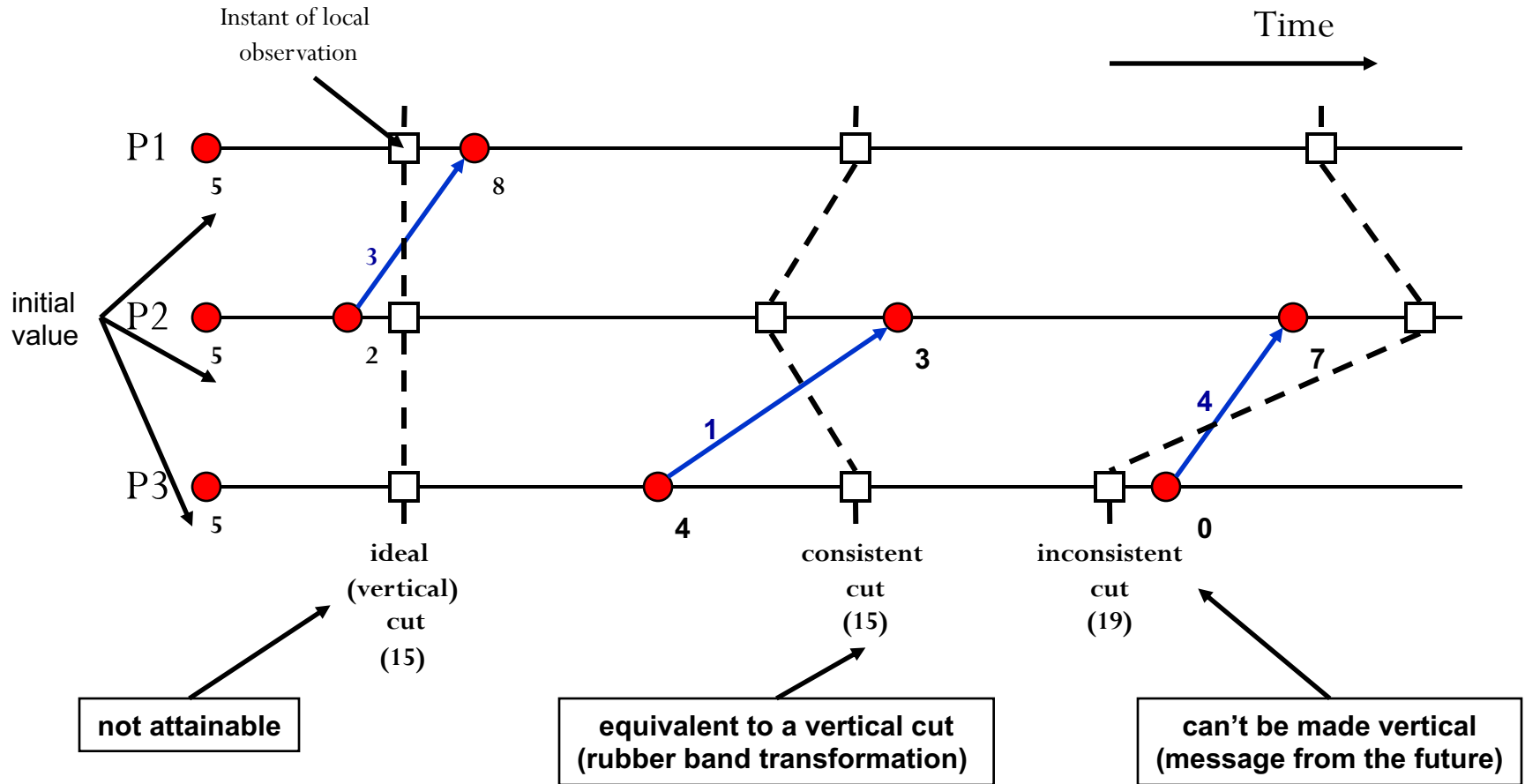
Simulate A Global State

- Recording the global state of a distributed system on-the-fly is an important paradigm.
 - Challenge: lack of globally shared memory, global clock and unpredictable message delays in a distributed system
- Notions of global time and global state closely related
- A process can (without *freezing* the whole computation) compute the *best possible approximation* of global state
- A global state that *could* have occurred
 - No process in the system can decide whether the state did really occur
 - Guarantee stable properties (*i.e.* once they become true, they remain true)

Consistent Cuts

- A cut (or time slice) is a zigzag line cutting a time diagram into 2 parts (past and future)
- E is augmented with a cut event c_i for each process $P_i: E' = E \cup \{c_i, \dots, c_n\} \therefore$
 - A cut C of an event set E is a finite subset $C \subseteq E: e \in C \wedge e' \prec_e \rightarrow e' \in C$
 - A cut C_1 is later than C_2 if $C_1 \supseteq C_2$
 - A consistent cut C of an event set E is a finite subset $C \subseteq E : e \in C \wedge e' \prec_e \rightarrow e' \in C$
 - i.e. a cut is consistent if every message received was previously sent (but not necessarily vice versa!)

Cuts (Summary)



“Rubber band transformation” changes metric, but keeps topology

System Model for Global Snapshots

- The system consists of a collection of n processes $p_1, p_2, \dots, p_n$ that are connected by channels.
- There are no globally shared memory and physical global clock and processes communicate by passing messages through communication channels.
- C_{ij} denotes the channel from process p_i to process p_j and its state is denoted by SC_{ij} .
- The actions performed by a process are modeled as three types of events:
 - Internal events, the message send event and the message receive event.
 - For a message m_{ij} that is sent by process p_i to process p_j , let $\text{send}(m_{ij})$ and $\text{rec}(m_{ij})$ denote its send and receive events.

Process States and Messages in transit

- At any instant, the state of process p_i , denoted by LS_i , is a result of the sequence of all the events executed by p_i till that instant.
- For an event e and a process state LS_i , $e \in LS_i$ iff e belongs to the sequence of events that have taken process p_i to state LS_i .
- For an event e and a process state LS_i , $e \notin LS_i$ iff e does not belong to the sequence of events that have taken process p_i to state LS_i .
- For a channel C_{ij} , the following set of messages can be defined based on the local states of the processes p_i and p_j

$$\text{Transit: } \text{transit}(LS_i, LS_j) = \{m_{ij} \mid \text{send}(m_{ij}) \in LS_i \vee \text{rec}(m_{ij}) \notin LS_j\}$$

Global States of Consistent Cuts

- The global state of a distributed system is a collection of the local states of the processes and the channels.
- A *global state* computed along a consistent cut is **correct**
- The *global state* of a consistent cut comprises the local state of each process at the time the cut event happens and the set of all messages sent but not yet received
- The *snapshot problem* consists in designing an efficient protocol which yields only consistent cuts and to collect the local state information
 - Messages crossing the cut must be captured
 - Chandy & Lamport presented an algorithm assuming that message transmission is FIFO

Chandy-Lamport Distributed Snapshot Algorithm

- Assumes FIFO communication in channels
- Uses a control message, called a marker to separate messages in the channels.
 - After a site has recorded its snapshot, it sends a marker, along all of its outgoing channels before sending out any more messages.
 - The marker separates the messages in the channel into those to be included in the snapshot from those not to be recorded in the snapshot.
- A process must record its snapshot no later than when it receives a marker on any of its incoming channels.
- The algorithm terminates after each process has received a marker on all of its incoming channels.
- All the local snapshots get disseminated to all other processes and all the processes can determine the global state.

Chandy-Lamport Distributed Snapshot Algorithm

Marker receiving rule for Process P_i

If (P_i has not yet recorded its state) *it*
 records its process state now
 records the state of c as the empty set
 turns on recording of messages arriving over other channels
else
 P_i records the state of c as the set of messages received over c
 since it saved its state

Marker sending rule for Process P_i

After P_i has recorded its state, for each outgoing channel c :
 P_i sends one marker message over c
 (before it sends any other message over c)

Chandy-Lamport Extensions: Spezialetti-Kerns and others

- Exploit concurrently initiated snapshots to reduce overhead of local snapshot exchange
 - Snapshot Recording
 - Markers carry identifier of initiator – first initiator recorded in a per process “*master*” variable.
 - **Region** - all the processes whose master field has same initiator.
 - Identifiers of concurrent initiators recorded in “*id-border-set*.”
 - Snapshot Dissemination
 - Forest of spanning trees is implicitly created in the system. Every Initiator is root of a spanning tree; nodes relay snapshots of rooted subtree to parent in spanning tree
 - Each initiator assembles snapshot for processes in its region and exchanges with initiators in adjacent regions.
- Others: multiple repeated snapshots; wave algorithm

Computing Global States without FIFO Assumption

- In a non-FIFO system, a marker cannot be used to delineate messages into those to be recorded in the global state from those not to be recorded in the global state.
- In a non-FIFO system, either some degree of inhibition or piggybacking of control information on computation messages to capture out-of-sequence messages.

Non-FIFO Channel Assumption: Lai-Yang Algorithm

- Emulates marker by using a coloring scheme
 - Every Process: White (before snapshot); Red (after snapshot).
 - Every message sent by a white (red) process is colored white (red) indicating if it was sent before(after) snapshot.
 - Each process (which is initially white) becomes red as soon as it receives a red message for the first time and starts a **virtual** broadcast algorithm to ensure that all processes will eventually become red
 - Get Dummy red messages to all processes (Flood neighbors)
- Determining Messages in transit
 - White process records history of white msgs sent/received on each channel.
 - When a process turns red, it sends these histories along with its snapshot to the initiator process that collects the global snapshot.
 - Initiator process evaluates $\text{transit}(LS_i, LS_j)$ to compute state of a channel C_{ij} :
 - $SC_{ij} = \text{white messages sent by } p_i \text{ on } C_{ij} - \text{white messages received by } p_j \text{ on } C_{ij} = \setminus \{ \text{send}(m_{ij}) \mid \text{send}(m_{ij}) \in LS_i \} - \{ \text{rec}(m_{ij}) \mid \text{rec}(m_{ij}) \in LS_j \} .$

Non-FIFO Channel Assumption: Termination Detection

- Required to detect that no white messages are in transit.
- Method 1: Deficiency Counting
 - Each process P_i keeps a counter $cntri$ that indicates the difference between the number of white messages it has sent and received before recording its snapshot.
 - It reports this value to the initiator process along with its snapshot and forwards all white messages, it receives henceforth, to the initiator.
 - Snapshot collection terminates when the initiator has received $\sum_i cntri$ number of forwarded white messages.
- Method 2
 - Each red message sent by a process carries a piggybacked value of the number of white messages sent on that channel before the local state recording.
 - Each process keeps a counter for the number of white messages received on each channel.
 - A process can detect termination of recording the states of incoming channels when it receives as many white messages on each channel as the value piggybacked on red messages received on that channel.

Non-FIFO Channel Assumption: Mattern Algorithm

- Uses Vector Clocks and assumes a single initiator
 - All process agree on some future **virtual time** s or a **set of virtual time instants** $s_1, \dots, s_n$ which are mutually concurrent and **did not yet** occur
 - A process takes its local snapshot at **virtual time** s
 - After time s the local snapshots are collected to construct a global snapshot
 - P_i ticks and then fixes its next time $s = C_i + (0, \dots, 0, 1, 0, \dots, 0)$ to be the common snapshot time
 - P_i broadcasts s
 - P_i blocks waiting for all the acknowledgements
 - P_i ticks again (setting $C_i = s$), takes its snapshot and broadcast a dummy message (i.e. force everybody else to advance their clocks to a value $\geq s$)
 - Each process takes its snapshot and sends it to P_i when its local clock becomes $\geq s$

Non-FIFO Channel Assumption: Mattern Algorithm

- Inventing a $n+1$ **virtual** process whose clock is managed by P_i
- P_i can use its clock and because the virtual clock C_{n+1} ticks only when P_i initiates a new run of snapshot :
 - The first n component of the vector can be omitted
 - The first broadcast phase is unnecessary
 - Counter modulo 2

Distributed Operating Systems - Introduction

Prof. Nalini Venkatasubramanian

(includes slides from Prof. Petru Eles and Profs. textbook slides by
Kshemkalyani/Singhal)

What does an OS do?

- Process/Thread Management
 - Scheduling
 - Communication
 - Synchronization
- Memory Management
- Storage Management
- FileSystems Management
- Protection and Security
- Networking

Operating System Types

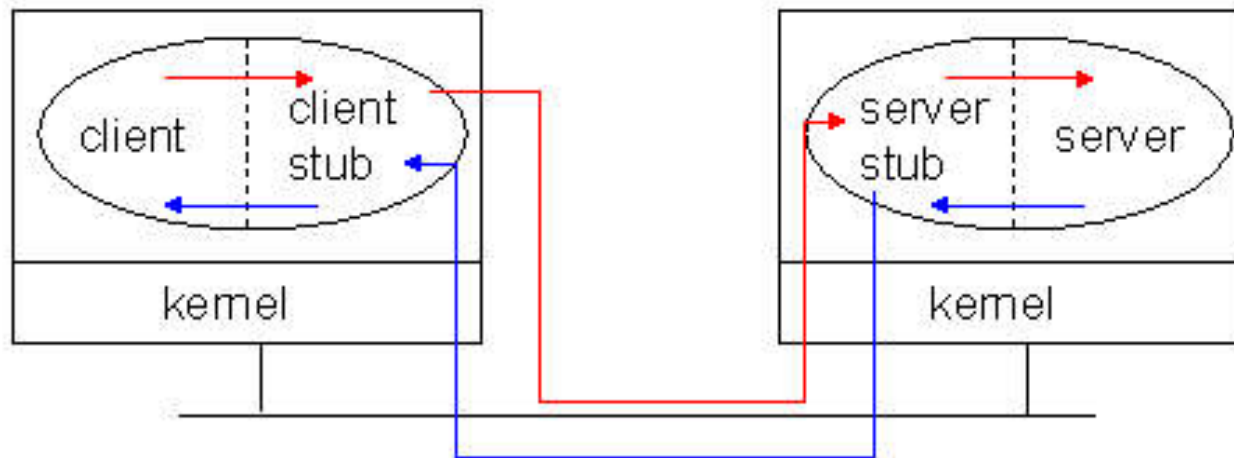
- Multiprocessor OS
 - Looks like a virtual uniprocessor, contains only one copy of the OS, communicates via shared memory, single run queue
- Network OS
 - Does not look like a virtual uniprocessor, contains n copies of the OS, communicates via shared files, n run queues
- Distributed OS
 - Looks like a virtual uniprocessor (more or less), contains n copies of the OS, communicates via messages, n run queues

Design Elements

- Communication
 - Two basic IPC paradigms used in DOS
 - Message Passing (RPC) and Shared Memory
 - synchronous, asynchronous
- Process Management
 - Process synchronization
 - Coordination of distributed processes is inevitable
 - mutual exclusion, deadlocks, leader election
 - Task Partitioning, allocation, load balancing, migration
- FileSystems
 - Naming of files/directories
 - File sharing semantics
 - Caching/update/replication

Remote Procedure Call

A convenient way to construct a client-server connection without explicitly writing send/ receive type programs (helps maintain transparency).



Remote Procedure Call (cont.)

- Client procedure **calls** the client stub in a normal way
- Client stub **builds** a message and **traps** to the kernel
- Kernel **sends** the message to remote kernel
- Remote kernel **gives** the message to server stub
- Server stub **unpacks** parameters and **calls** the server
- Server **computes** results and **returns** it to server stub
- Server stub **packs** results in a message and **traps** to kernel
- Remote kernel **sends** message to client kernel
- Client kernel **gives** message to client stub
- Client stub **unpacks** results and **returns** to client

Distributed Shared Memory

- Provides a shared-memory abstraction in the loosely coupled distributed-memory processors.
- Issues
 - Granularity of the block size
 - Synchronization
 - Memory Coherence (Consistency models)
 - Data Location and Access
 - Replacement Strategies
 - Thrashing
 - Heterogeneity

Distributed Mutual Exclusion

- Mutual exclusion
 - ensures that concurrent processes have serialized access to shared resources - the *critical section problem*.
 - At any point in time, only one process can be executing in its critical section.
- Shared variables (semaphores) cannot be used in a distributed system
 - Mutual exclusion must be based on message passing, in the context of unpredictable delays and incomplete knowledge
 - In some applications (e.g. transaction processing) the resource is managed by a server which implements its own lock along with mechanisms to synchronize access to the resource.

Approaches to Distributed Mutual Exclusion

- Central coordinator based approach
 - A centralized coordinator determines who enters the CS
- Distributed approaches to mutual exclusion
 - Token based approach
 - A unique token is shared among the sites. A site is allowed to enter its CS if it possesses the token.
 - Mutual exclusion is ensured because the token is unique.
 - Non-token based approach
 - Two or more successive rounds of messages are exchanged among the sites to determine which site will enter the CS next.
 - Quorum based approach
 - Each site requests permission to execute the CS from a subset of sites (called a quorum).
 - Any two quorums contain a common site. This common site is responsible to make sure that only one request executes the CS at any time.

Requirements/Conditions

- Safety Property (*Mutual Exclusion*)
 - At any instant, only one process can execute the critical section.
- Liveness Property (*Progress*)
 - This property states the absence of deadlock and starvation. Two or more sites should not endlessly wait for messages which will never arrive.
- Fairness (*Bounded Waiting*)
 - Each process gets a fair chance to execute the CS. Fairness property generally means the CS execution requests are executed in the order of their arrival (time is determined by a logical clock) in the system.

Performance Metrics for Mutual Exclusion Algorithms

- **Message complexity**
 - The number of messages required per CS execution by a site.
- **Synchronization delay**
 - After a site leaves the CS, it is the time required before the next site enters the CS
- **Response time**
 - The time interval a request waits for its CS execution to be over after its request messages have been sent out
- **System throughput**
 - The rate at which the system executes requests for the CS.

$$\text{System throughput} = 1 / (\text{SD} + E)$$

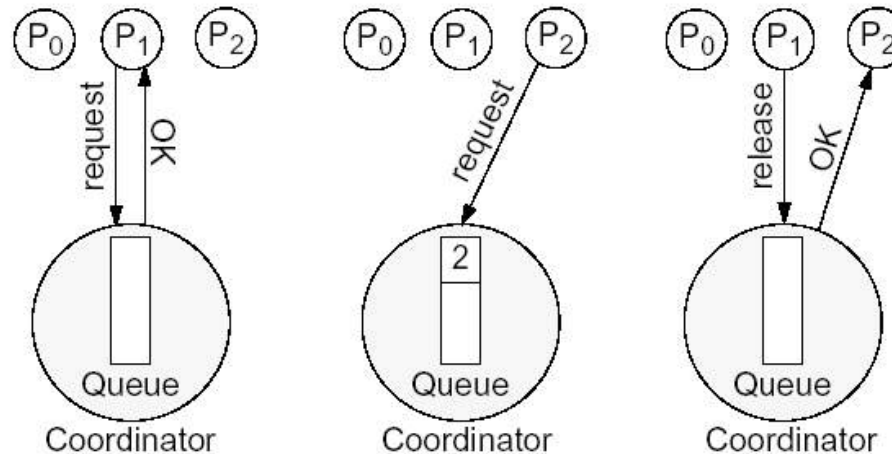
where SD is the synchronization delay and E is the average critical section execution time

Mutual Exclusion Techniques Covered

- Central Coordinator Algorithm
- In a distributed environment it seems more natural to implement mutual exclusion, based upon distributed agreement - not on a central coordinator.
- Distributed Non-token based (Timestamp-Based Algorithms)
 - Lamport's Algorithm
 - Ricart-Agrawala1 Algorithm
 - Variation – Quorum based (Maekawa's Algorithm)
- Distributed Token Based
 - Ricart-Agrawala Second Algorithm
 - Token Ring Algorithm

Central Coordinator Algorithm

☛ A central coordinator grants permission to enter a CS.



- To enter a CS, a process sends a request message to the coordinator and then waits for a reply (during this waiting period the process can continue with other work).
- The reply from the coordinator gives the right to enter the CS.
- After finishing work in the CS the process notifies the coordinator with a release message.

Lamport's Algorithm

- Basic Idea
 - Requests for CS are executed in the increasing order of timestamps and time is determined by logical clocks.
 - Every site S_i keeps a queue, request queue_i, which contains mutual exclusion requests ordered by their timestamps.
 - This algorithm requires communication channels to deliver messages the FIFO order.

Lamport's Algorithm

- Requesting the critical section
 - When a site S_i wants to enter the CS, it broadcasts a $\text{REQUEST}(ts_i, i)$ message to all other sites and places the request on request queue i . ((ts_i, i) denotes the timestamp of the request.)
 - When a site S_j receives the $\text{REQUEST}(ts_i, i)$ message from site S_i , it places site S_i 's request on request queue of j and returns a timestamped REPLY message to S_i
- Executing the critical section
 - Site S_i enters the CS when the following two conditions hold:
 - L1: S_i has received a message with timestamp larger than (ts_i, i) from all other sites.
 - L2: S_i 's request is at the top of request queue i .
- Releasing the critical section
 - Site S_i , upon exiting the CS, removes its request from the top of its request queue and broadcasts a timestamped RELEASE message to all other sites.
 - When a site S_j receives a RELEASE message from site S_i , it removes S_i 's request from its request queue.
 - When a site removes a request from its request queue, its own request may come at the top of the queue, enabling it to enter the CS.

Performance – Lamport's Algorithm

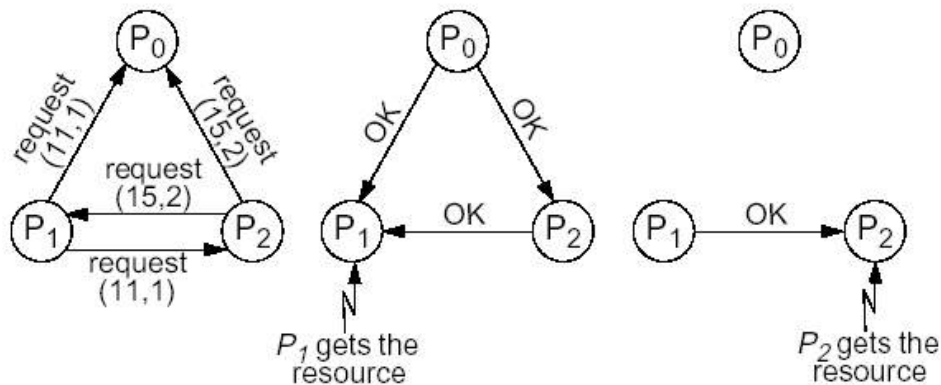
- For each CS execution Lamport's algorithm requires
 - $(N - 1)$ REQUEST messages, $(N - 1)$ REPLY messages, and $(N - 1)$ RELEASE messages.
 - Thus, Lamport's algorithm requires $3(N - 1)$ messages per CS invocation.
- Optimization
 - In Lamport's algorithm, REPLY messages can be omitted in certain situations.
 - For example, if site S_j receives a REQUEST message from site S_i after it has sent its own REQUEST message with timestamp higher than the timestamp of site S_i 's request, then site S_j need not send a REPLY message to site S_i .
 - This is because when site S_i receives site S_j 's request with timestamp higher than its own, it can conclude that site S_j does not have any smaller timestamp request which is still pending.
 - With this optimization, Lamport's algorithm requires between $3(N - 1)$ and $2(N - 1)$ messages per CS execution.

Ricart-Agrawala Algorithm

- Uses only two types of messages – REQUEST and REPLY.
- It is assumed that all processes keep a (Lamport's) logical clock which is updated according to the clock rules.
 - The algorithm requires a total ordering of requests. Requests are ordered according to their global logical timestamps; if timestamps are equal, process identifiers are compared to order them.
- The process that requires entry to a CS multicasts the request message to all other processes competing for the same resource.
 - Process is allowed to enter the CS when all processes have replied to this message.
 - The request message consists of the requesting process' timestamp (logical clock) and its identifier.
- Each process keeps its state with respect to the CS: released, requested, or held.

Ricart-Agrawala Algorithm (cont'd)

- A request issued by a process P_j is blocked by another process P_i only if P_i is holding the resource or if it is requesting the resource with a higher priority (this means a smaller timestamp) than P_j .



Problems

- The algorithm is expensive in terms of message traffic; it requires $2(n-1)$ messages for entering a CS: $(n-1)$ requests and $(n-1)$ replies.
- The failure of any process involved makes progress impossible if no special recovery measures are taken.

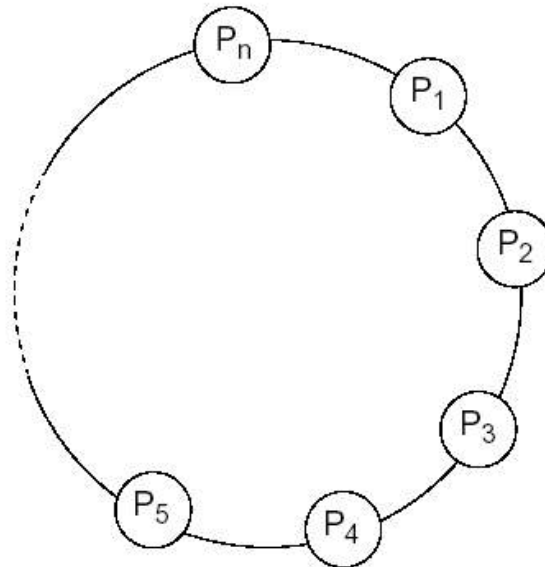


Quorum-Based Consensus – Maekawa's Algorithm

- Site obtains permission only from a subset of sites to enter CS
- Multicasts messages to a voting subset of processes'
 - Each process p_i is associated with a voting set v_i (of processes)
 - Each process belongs to its own voting set
 - The intersection of any two voting sets is non-empty
 - Each voting set is of size K
 - Each process belongs to M other voting sets
 - To access a critical section, p_i requests permission from all other processes in its own voting set v_i
 - Voting set member gives permission to only one requestor at a time, and queues all other requests
 - Guarantees safety
 - May not guarantee liveness (may deadlock)
 - Maekawa showed that $K=M=\sqrt{N}$ works best
 - One way of doing this is to put N processes in a $\sqrt{N}$ by $\sqrt{N}$ matrix and take union of row & column containing p_i as its voting set.

Token Ring Algorithm

- A very simple way to solve mutual exclusion $\Rightarrow$ arrange the n processes $P_1, P_2, \dots, P_n$ in a logical ring.
- The logical ring topology is created by giving each process the address of one other process which is its neighbour in the clockwise direction.
- The logical ring topology is unrelated to the physical interconnections between the computers.

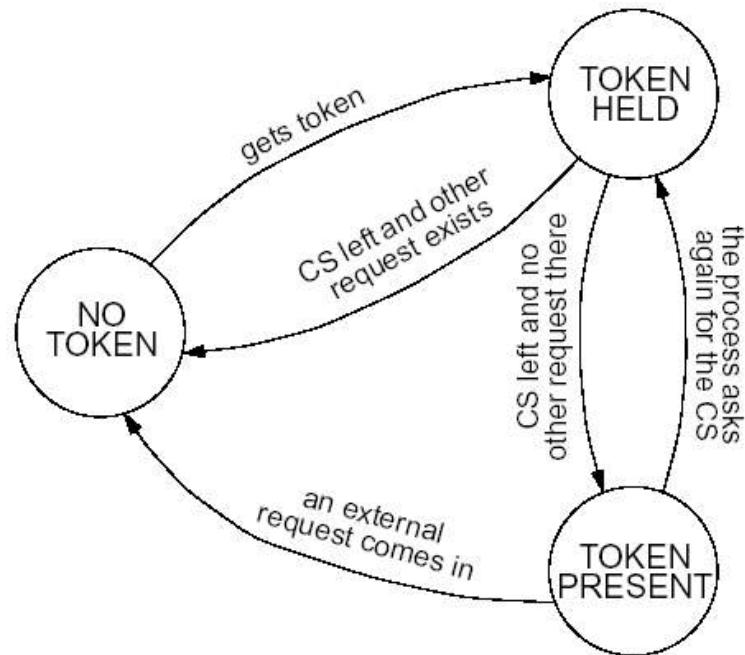


Ricart-Agrawala Second Algorithm

- A process is allowed to enter the critical section when it gets the token.
 - Initially the token is assigned arbitrarily to one of the processes.
- In order to get the token it sends a request to all other processes competing for the same resource.
 - The request message consists of the requesting process' timestamp (logical clock) and its identifier.
- When a process P_i leaves a critical section
 - it passes the token to one of the processes which are waiting for it; this will be the first process P_j , where j is searched in order $[i+1, i+2, \dots, n, 1, 2, \dots, i-2, i-1]$ for which there is a pending request.
 - If no process is waiting, P_i retains the token (and is allowed to enter the CS if it needs); it will pass over the token as result of an incoming request.
- How does P_i find out if there is a pending request?
 - Each process P_i records the timestamp corresponding to the last request it got from process P_j , in request $P_i[j]$. In the token itself, $token[j]$ records the timestamp (logical clock) of P_j 's last holding of the token. If $requestP_i[j] > token[j]$ then P_j has a pending request.

Ricart-Agrawala Second Algorithm (cont'd)

- Each process keeps its state with respect to the token:
NO-TOKEN, TOKEN-PRESENT, TOKEN-HOLD.



Suzuki-Kazami Broadcast Algorithm

- If a site wants to enter the CS and it does not have the token, it broadcasts a REQUEST message for the token to all other sites.
- A site which possesses the token sends it to the requesting site upon the receipt of its REQUEST message.
- If a site receives a REQUEST message when it is executing the CS, it sends the token only after it has completed the execution of the CS.
- Two Issues
 - Outdated Requests: Due to variable message delays, site may receive token request message after request has been satisfied. Token to outdated requestor results in poor performance
 - When a process is done, which of the outstanding requests should it satisfy?

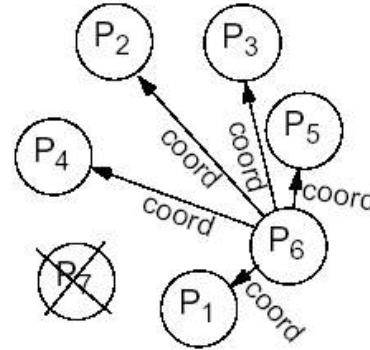
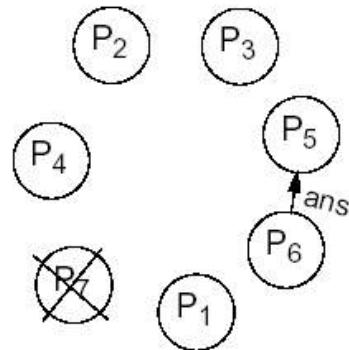
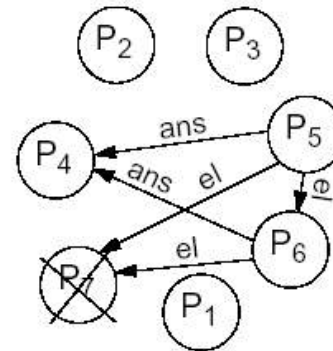
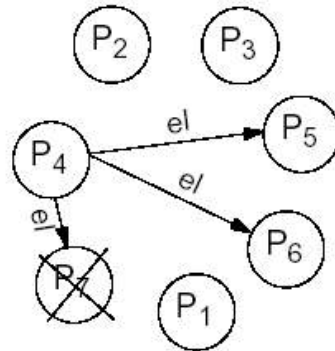
Election Algorithms

- It doesn't matter which process is elected.
 - What is important is that one and only one process is chosen (we call this process the coordinator) and all processes agree on this decision.
- Assume that each process has a unique number (identifier).
 - In general, election algorithms attempt to locate the process with the highest number, among those which currently are up.
- Election is typically started after a failure occurs.
 - The detection of a failure (e.g. the crash of the current coordinator) is normally based on time-out → a process that gets no response for a period of time suspects a failure and initiates an election process.
- An election process is typically performed in two phases:
 - Select a leader with the highest priority.
 - Inform all processes about the winner.

The Bully Algorithm

- A process has to know the identifier of all other processes
 - (it doesn't know, however, which one is still up); the process with the highest identifier, among those which are up, is selected.
- Any process could fail during the election procedure.
- When a process P_i detects a failure and a coordinator has to be elected
 - it sends an election message to all the processes with a higher identifier and then waits for an answer message:
 - If no response arrives within a time limit
 - P_i becomes the coordinator (all processes with higher identifier are down)
 - it broadcasts a coordinator message to all processes to let them know.
 - If an answer message arrives,
 - P_i knows that another process has to become the coordinator → it waits in order to receive the coordinator message.
 - If this message fails to arrive within a time limit (which means that a potential coordinator crashed after sending the answer message) P_i resends the election message.
- When receiving an election message from P_i
 - a process P_j replies with an answer message to P_i and
 - then starts an election procedure itself(unless it has already started one) it sends an election message to all processes with higher identifier.
- Finally all processes get an answer message, except the one which becomes the coordinator.

The Bully Algorithm (cont'd)



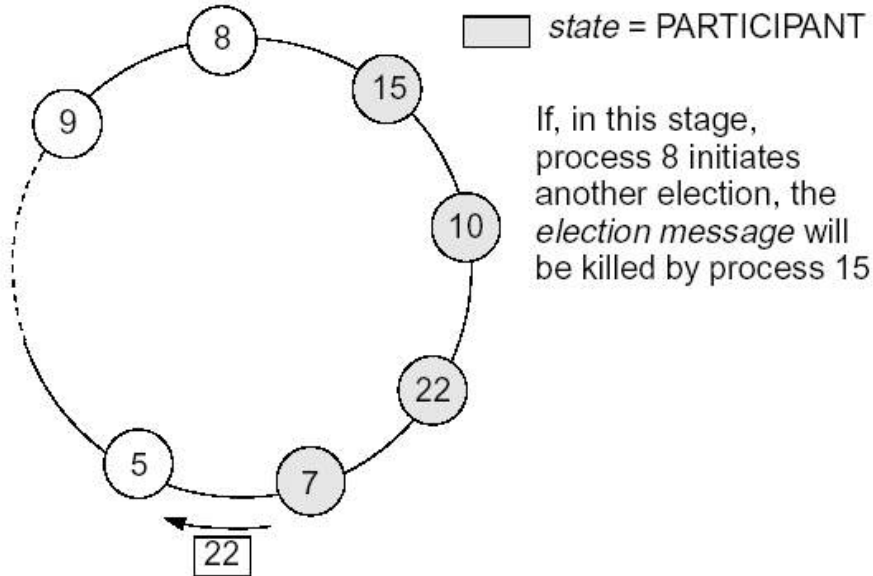
- If P_6 crashes before sending the coordinator message, P_4 and P_5 restart the election process.
- ☞ The best case: the process with the second highest identifier notices the coordinator's failure. It can immediately select itself and then send $n-2$ coordinator messages.
- ☞ The worst case: the process with the lowest identifier initiates the election; it sends $n-1$ election messages to processes which themselves initiate each one an election $\Rightarrow O(n^2)$ messages.



The Ring-based Algorithm

- We assume that the processes are arranged in a logical ring
 - Each process knows the address of one other process, which is its neighbor in the clockwise direction.
- The algorithm elects a single coordinator, which is the process with the highest identifier.
- Election is started by a process which has noticed that the current coordinator has failed.
 - The process places its identifier in an election message that is passed to the following process.
 - When a process receives an election message
 - It compares the identifier in the message with its own.
 - If the arrived identifier is greater, it forwards the received election message to its neighbor
 - If the arrived identifier is smaller it substitutes its own identifier in the election message before forwarding it.
 - If the received identifier is that of the receiver itself → this will be the coordinator.
- The new coordinator sends an elected message through the ring.

The Ring-Based Algorithm (cont'd)



☞ With one single election started:

- On average: $n/2$ (election) messages needed to reach maximal node; n (election) messages to return to maximal node; n messages to rotate *elected message*.
Number of messages: $2n + n/2$.
- Worst case: $n-1$ messages needed to reach maximal node;
Number of messages: $3n - 1$.

☞ The ring algorithm is more efficient on average than the bully algorithm.



Distributed Deadlocks

- Deadlocks is a fundamental problem in distributed systems.
- A process may request resources in any order, which may not be known a priori and a process can request resource while holding others.
- If the sequence of the allocations of resources to the processes is not controlled, deadlocks can occur.
- A deadlock is a state where a set of processes request resources that are held by other processes in the set.
- Conditions for a deadlocks
 - Mutual exclusion, hold-and-wait, No-preemption and circular wait.

Modeling Deadlocks

- In addition to the standard assumptions (no shared memory, no global clock, no failures), we make the following assumptions:
- The systems have only reusable resources.
- Processes are allowed to make only exclusive access to resources.
- There is only one copy of each resource.
- A process can be in two states: *running* or *blocked*.
 - In the running state (also called *active state*), a process has all the needed resources and is either executing or is ready for execution.
 - In the blocked state, a process is waiting to acquire some resource.
- The state of the system can be modeled by directed graph, called a *wait for graph* (WFG).
 - In a WFG , nodes are processes and there is a directed edge from node P1 to mode P2 if P1 is blocked and is waiting for P2 to release some resource.
- A system is deadlocked if and only if there exists a directed cycle or knot in the WFG.

Techniques for Handling Deadlocks

- Note: No site has accurate knowledge of the current state of the system
- Techniques
 - Deadlock Prevention (collective/ordered requests, preemption)
 - Inefficient, impractical
 - Deadlock Avoidance
 - A resource is granted to a process if the resulting global system state is safe
 - Requires advance knowledge of processes and their resource requirements
 - Impractical
 - Deadlock Detection and Recovery
 - Maintenance of local/global WFG and searching of the WFG for the presence of cycles (or knots), local/centralized deadlock detectors
 - Recovery by operator intervention, break wait-for dependencies, termination and rollback

Classes of Deadlock Detection Algorithms

- Path-pushing
 - distributed deadlocks are detected by maintaining an explicit global WFG (constructed locally and pushed to neighbors)
- Edge-chasing (single resource model, AND model)
 - the presence of a cycle in a distributed graph structure is be verified by propagating special messages called probes, along the edges of the graph. The formation of cycle can be detected by a site if it receives the matching probe sent by it previously.
- Diffusion computation (OR model, AND-OR model)
 - deadlock detection computation is diffused through the WFG of the system.
- Global state detection (Unrestricted, P-out-of-Q model)
 - Take a snapshot of the system and examining it for the condition of a deadlock.

Process Management

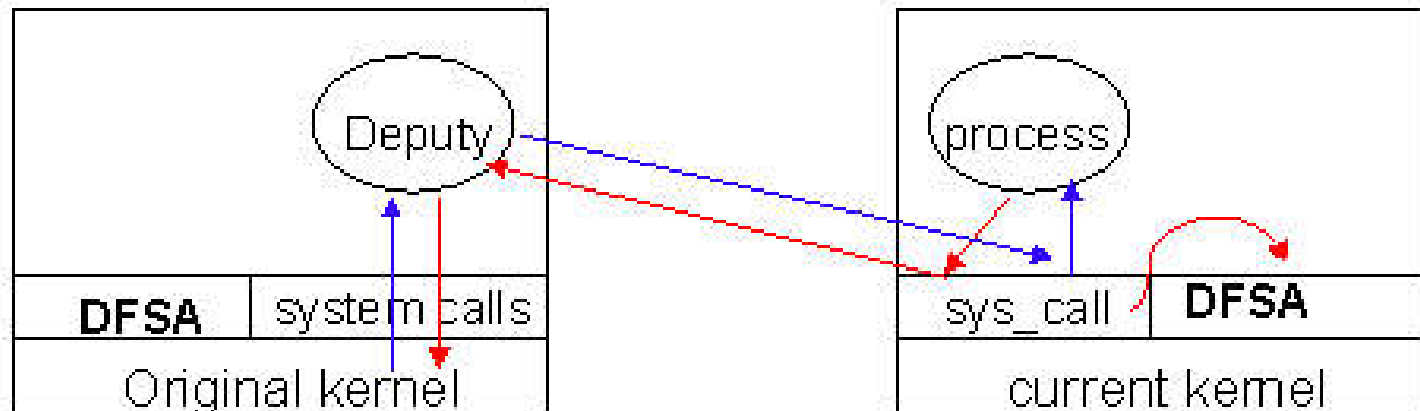
- Process migration
 - Freeze the process on the source node and restart it at the destination node
 - Transfer of the process address space
 - Forwarding messages meant for the migrant process
 - Handling communication between cooperating processes separated as a result of migration
 - Handling child processes
- Process migration in heterogeneous systems

Mosix: File Access

Each file access must go back to deputy...

= = Very Slow for I/O apps.

Solution: Allow processes to access a distributed file system through the current kernel.



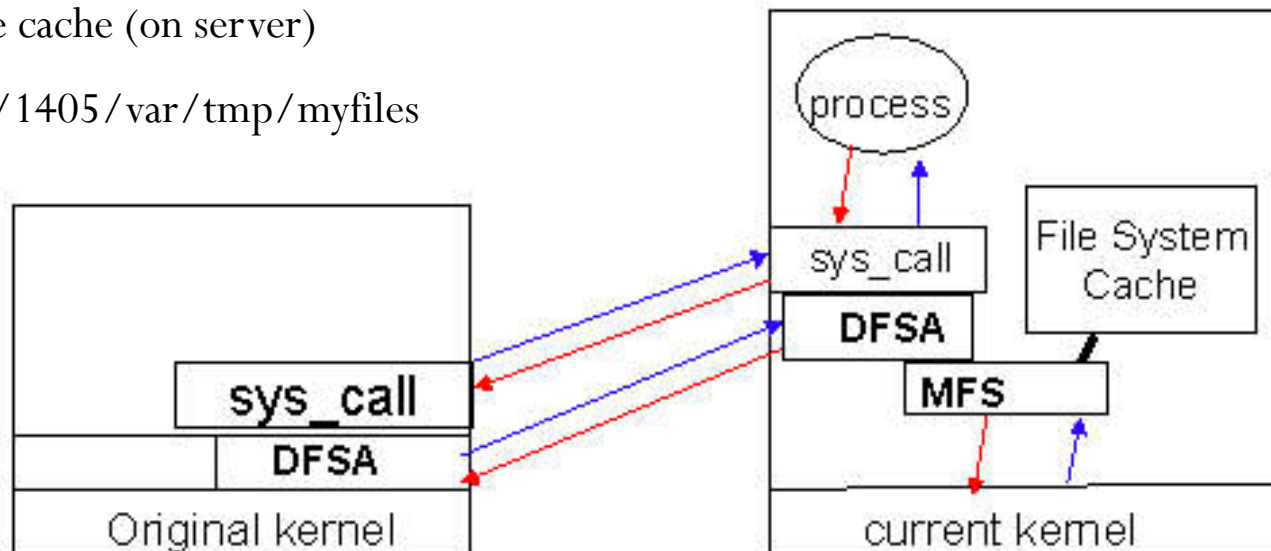
Mosix: File Access

- **DFSA**

- Requirements (cache coherent, monotonic timestamps, files not deleted until all nodes finished)
- Bring the process to the files.

- **MFS**

- Single cache (on server)
- /mfs/1405/var/tmp/myfiles



Dynamic Load Balancing

- Dynamic Load Balancing on Highly Parallel Computers
 - *Seek to minimize total execution time of a single application running in parallel on a multiprocessor system*
 - Sender Initiated Diffusion (SID), Receiver Initiated Diffusion(RID), Hierarchical Balancing Method (HBM), Gradient Model (GM), Dynamic Exchange method (DEM)
- Dynamic Load Balancing on Web Servers
 - *Seek to improve response time using distributed web-server architectures , by scheduling client requests among multiple nodes in a transparent way*
 - Client-based approach, DNS-Based approach, Dispatcher-based approach, Server-based approach
- Dynamic Load Balancing on Multimedia Servers
 - *Aim to maximize requests and preserve QoS for admitted requests by adaptively scheduling requests given knowledge of where objects are placed*
 - Adaptive Scheduling of Video Objects, Predictive Placement of Video Objects

Distributed File Systems (DFS)

- DFS is a distributed implementation of the classical file system model
 - Issues - File and directory naming, semantics of file sharing
- Important features of DFS
 - Transparency, Fault Tolerance
- Implementation considerations
 - caching, replication, update protocols
- The general principle of designing DFS: know the clients have cycles to burn, cache whenever possible, exploit usage properties, minimize system wide change, trust the fewest possible entries and batch if possible.

File Sharing Semantics

- One-copy semantics
 - Updates are written to the single copy and are available immediately
- Serializability
 - Transaction semantics (file locking protocols implemented - share for read, exclusive for write).
- Session semantics
 - Copy file on open, work on local copy and copy back on close

Example: Sun-NFS

- Supports heterogeneous systems
- Architecture
 - Server exports one or more directory trees for access by remote clients
 - Clients access exported directory trees by mounting them to the client local tree
 - Diskless clients mount exported directory to the root directory
- Protocols
 - Mounting protocol
 - Directory and file access protocol - stateless, no open-close messages, full access path on read/write
- Semantics - no way to lock files

Example: Andrew File System

- Supports information sharing on a large scale
- Uses a session semantics
 - Entire file is copied to the local machine (Venus) from the server (Vice) when open. If file is changed, it is copied to server when closed.
 - Works because in practice, most files are changed by one person

The Coda File System

- Descendant of AFS that is substantially more resilient to server and network failures.
- Support for “mobile” users.
- Directories are replicated in several servers (Vice)
- When the Venus is disconnected, it uses local versions of files. When Venus reconnects, it reintegrates using optimistic update scheme.

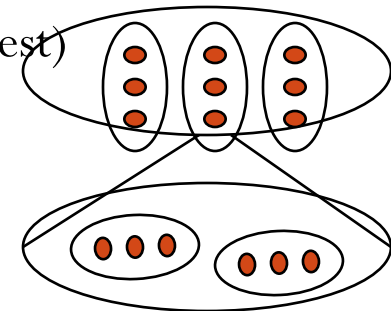
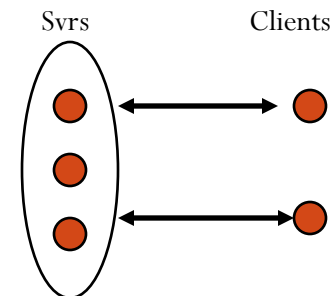
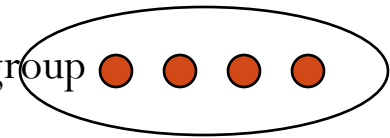
Messaging and Group Communication

ICS 230 Distributed Systems

(with some slides modified from S.Ghosh's classnotes)

What type of group communication ?

- Open group (anyone can join, customers of Walmart)
- Closed groups (closed membership, class of 2000)
- Peer
 - All members are equal, All members send messages to the group
 - All members receive all the messages
 - E.g. UCI students, UCI faculty
- Client-Server
 - Common communication pattern
 - replicated servers
 - Client may or may not care which server answers
- Diffusion group
 - Servers sends to other servers and clients
- Hierarchical (one or more members are diff. from the rest)
 - Highly and easy scalable



Multicast

- Basic Multicast: Does not consider failures
 - Liveness: Each process must receive every message
 - Integrity : No spurious message received
 - No duplicates: Accepts exactly one copy of a message
- Reliable multicast: tolerates (certain kinds of) failures.
- Atomic Multicast:
 - A multicast is **atomic**, when the message is delivered to **every correct** member, or to **no member** at all.
 - In general, processes may crash, yet the atomicity of the multicast is to be guaranteed.
- Reliable Atomic Multicast
 - Scalability a key issue

Steiner Trees and Core Based Trees

- Given a weighted graph (N, L) and a subset N' in N , identify a subset L' in L such that (N', L') is a subgraph of (N, L) that connects all the nodes of N' .
 - A minimal Steiner tree is a minimal weight subgraph $(N'; L')$.
 - NP-complete ; need heuristics
- Core-based Trees
 - Multicast tree constructed dynamically, grows on demand.
 - Each group has a core node(s)
 - A node wishing to join the tree as a receiver sends a unicast join message to the core node.
 - The join marks the edges as it travels; it either reaches the core node, or some node already part of the tree. The path followed by the join till the core/multicast tree is grafted to the multicast tree.
 - A node on the tree multicasts a message by using flooding on the core tree.
 - A node not on the tree sends a message towards the core node; as soon as the message reaches any node on the tree, it is flooded on the tree.

Using Traditional Transport Protocols

- TCP/IP
 - Automatic flow control, reliable delivery, connection service, complexity
 - linear degradation in performance
- Unreliable broadcast/multicast
 - UDP, IP-multicast - assumes h/w support
 - IP-multicast
 - A bandwidth-conserving technology where the router reduces traffic by replicating a single stream of information and forwarding them to multiple clients.
 - Sender sends a single copy to a special multicast IP address (Class D) that represents a group, where other members register.
 - message losses high(30%) during heavy load
 - Reliable IP-multicast very expensive

Group Communication Issues

- Ordering
- Delivery Guarantees
- Membership
- Failure

Ordering Service

- Unordered
- Single-Source FIFO (SSF)
 - For all message r_1 and all object j , r_1 is received at j before r_2 is received at j if r_1 is sent before r_2 .
- Totally Ordered
 - For all message r_1 and all object j , r_1 is received at j before r_2 is received at j if r_1 is sent before r_2 .
- Causally Ordered
 - For all message r_1 and all object j , r_1 happens before r_2 if r_1 is sent before r_2 and r_1 is received at j before r_2 is received at j .

Delivery guarantees

- Agreed Delivery
 - guarantees total order of message delivery and allows a message to be delivered as soon as all of its predecessors in the total order have been delivered.
- Safe Delivery
 - requires in addition, that if a message is delivered by the GC to any of the processes in a configuration, this message has been received and will be delivered to each of the processes in the configuration unless it crashes.

Membership

- Messages addressed to the group are received by all group members
- If processes are added to a group or deleted from it (due to process crash, changes in the network or the user's preference), need to report the change to all active group members, while keeping consistency among them
- Every message is delivered in the context of a certain configuration, which is not always accurate. However, we may want to guarantee
 - Failure atomicity
 - Uniformity
 - Termination

Some GC Properties

- Atomic Multicast
 - Message is delivered to all processes or to none at all. May also require that messages are delivered in the same order to all processes.
- Failure Atomicity
 - Failures do not result in incomplete delivery of multicast messages or holes in the causal delivery order
- Uniformity
 - A view change reported to a member is reported to all other members
- Liveness
 - A machine that does not respond to messages sent to it is removed from the local view of the sender within a finite amount of time.

Virtual Synchrony

- Virtual Synchrony
 - Introduced in ISIS, orders group membership changes along with the regular messages
 - Ensures that failures do not result in incomplete delivery of multicast messages or holes in the causal delivery order([failure atomicity](#))
 - Ensures that, if two processes observe the same two consecutive membership changes, receive the [same set](#) of regular multicast messages between the two changes
 - A view change acts as a barrier across which no multicast can pass
 - Does not constrain the behavior of faulty or isolated processes

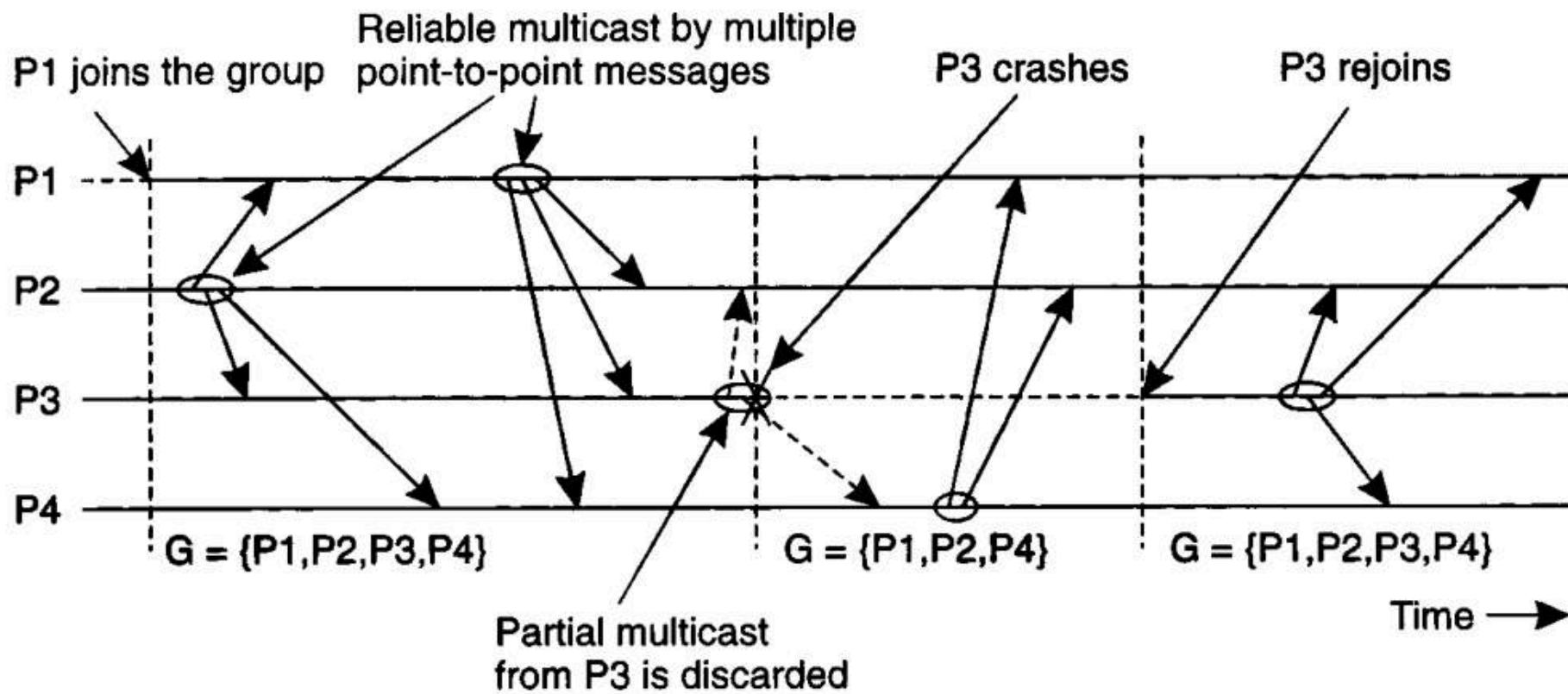
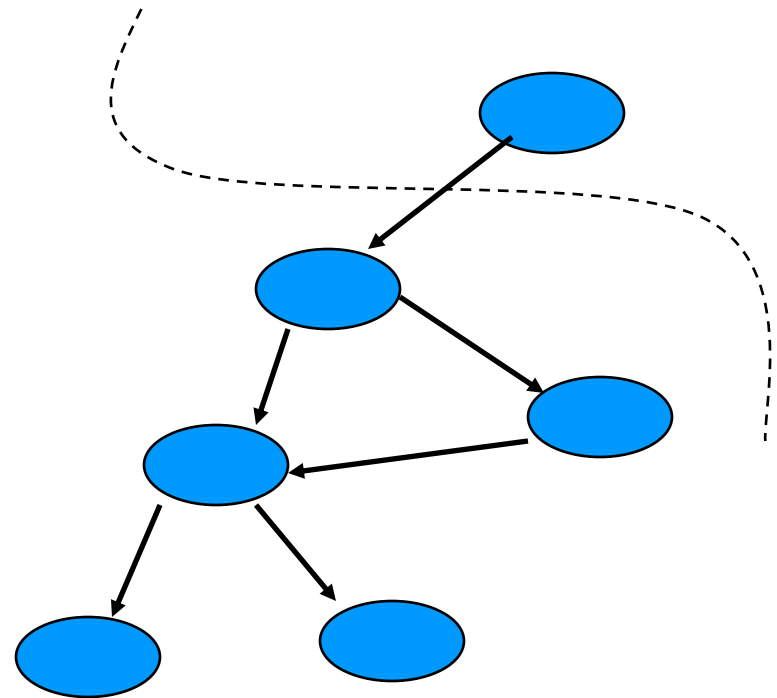


Figure 7-12. The principle of virtual synchronous multicast.

Faults and Partitions

- When detecting a processor P from which we did not hear for a certain timeout, we issue a fault message
- When we get a fault message, we adopt it (and issue our copy)
- **Problem:** maybe P is only slow
- When a partition occurs, we can not always completely determine who received which messages (**there is no solution to this problem**)



Extended Virtual Synchrony(cont.)

- Virtual synchrony handles recovered processes as new processes
 - Can cause inconsistencies with network partitions
- Network partitions are real
 - Gateways, bridges, wireless communication

Extended Virtual Synchrony Model

- Network may partition into finite number of components
 - Two or more may merge to form a larger component
- Each membership with a unique identifier is a *configuration*.
 - Membership ensures that all processes in a configuration agree on the membership of that configuration

Regular and Transitional Configurations

- To achieve safe delivery with partitions and remerges, the EVS model defines:
 - Regular Configuration
 - New messages are broadcast and delivered
 - Sufficient for FIFO and causal communication modes
 - Transitional Configuration
 - No new messages are broadcast, only remaining messages from prior regular configuration are delivered.
- Regular configuration may be followed and preceded by several transitional configurations.

Totem



- Provides a Reliable totally ordered multicast service over LAN
- Intended for complex applications in which fault-tolerance and soft real-time performance are critical
 - High throughput and low predictable latency
 - Rapid detection of, and recovery from, faults
 - System wide total ordering of messages
 - Scalable via hierarchical group communication
 - Exploits [hardware broadcast](#) to achieve high-performance
- Provides 2 delivery services
 - Agreed
 - Safe
- Use timestamp to ensure total order and sequence numbers to ensure reliable delivery

ISIS



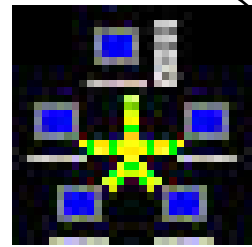
- Tightly coupled distributed system developed over loosely coupled processors
- Provides a toolkit mechanism for distributing programming, whereby a DS is built by interconnecting fairly conventional non-distributed programs, using tools drawn from the kit
- Define
 - how to create, join and leave a group
 - group [membership](#)
 - [virtual synchrony](#)
- Initially point-to-point (TCP/IP)
- Fail-stop failure model

Horus



- Aims to provide a very flexible environment to configure group of protocols specifically adapted to problems at hand
- Provides efficient support for virtual synchrony
- Replaces point-to-point communication with group communication as the fundamental abstraction, which is provided by stacking protocol modules that have a **uniform (upcall, downcall) interface**
- **Not every sort of protocol blocks make sense**
- HCPI
 - Stability of messages
 - membership
- Electra
 - CORBA-Compliant interface
 - method invocation transformed into multicast

Transis



- How different components of a partitioned network can operate autonomously and then merge operations when they become reconnected ?
- Are different protocols for fast-local and slower-cluster communication needed ?
- A large-scale multicast service designed with the following goals
 - Tackling network partitions and providing tools for recovery from them
 - Meeting needs of large networks through hierarchical communication
 - Exploiting fast-clustered communication using IP-Multicast
- Communication modes
 - FIFO
 - Causal
 - Agreed
 - Safe

Fault Tolerant Distributed Systems

ICS 230

Prof. Nalini Venkatasubramanian

(with some slides modified from Prof. Ghosh,
University of Iowa and Indranil Gupta, UIUC)

Classification of failures

Crash failure

Security failure

Omission failure

Temporal failure

Transient failure

Byzantine failure

Software failure

Environmental perturbations

Crash failures

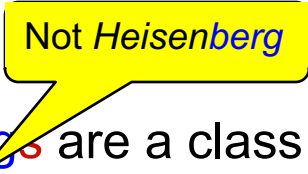
Crash failure = the process halts. It is *irreversible*.

In synchronous system, it is easy to detect crash failure (using *heartbeat signals* and *timeout*). But in asynchronous systems, it is never accurate, since it is *not possible* to distinguish between a process that has crashed, and a process that is running *very slowly*.

Some failures may be complex and nasty. **Fail-stop failure** is a *simple abstraction* that *mimics* crash failure when program execution becomes arbitrary. Implementations help detect which processor has failed. If a system cannot tolerate fail-stop failure, then it cannot tolerate crash.

Transient failure

(Hardware) Arbitrary perturbation of the global state. May be induced by power surge, weak batteries, lightning, radio-frequency interferences, cosmic rays etc.



Not *Heisenberg*

(Software) Heisenbugs are a class of temporary internal faults and are intermittent. They are essentially permanent faults whose conditions of activation occur rarely or are not easily reproducible, so they are harder to detect during the testing phase.

Over 99% of bugs in IBM DB2 production code are non-deterministic and transient (Jim Gray)

Temporal failures

Inability to meet deadlines – correct results are generated, but too late to be useful. Very important in real-time systems.

May be caused by poor algorithms, poor design strategy or loss of synchronization among the processor clocks

Byzantine failure

Anything goes! Includes every conceivable form of erroneous behavior. The weakest type of failure

Numerous possible causes. Includes **malicious behaviors** (like a process executing a different program instead of the specified one) **too**.

Most difficult kind of failure to deal with.

Hardware Errors and Error Control Schemes

Failures	Causes	Metric s	Traditional Approaches
Soft Errors, Hard Failures, System Crash	External Radiations, Thermal Effects, Power Loss, Poor Design, Aging	FIT, MTTF, MTBF	Spatial Redundancy (TMR, Duplex, RAID-1 etc.) and Data Redundancy (EDC, ECC, RAID-5, etc.)

❑ Hardware failures are increasing as technology scales

❑ (e.g.) SER increases by up to 1000 times [Mastipuram, 04]

❑ Redundancy techniques are expensive

❑ (e.g.) ECC-based protection in caches can incur 95% performance penalty [Li, 05]

- FIT: Failures in Time (10^9 hours)
- MTTF: Mean Time To Failure
- MTBF: Mean Time b/w Failures
- TMR: Triple Modular Redundancy
- EDC: Error Detection Codes
- ECC: Error Correction Codes
- RAID: Redundant Array of Inexpensive Drives

Software Errors and Error Control Schemes

Failures	Causes	Metrics	Traditional Approaches
Wrong outputs, Infinite loops, Crash	Incomplete Specification, Poor software design, Bugs, Unhandled Exception	Number of Bugs/Klines, QoS, MTTF, MTBF	Spatial Redundancy (N-version Programming, etc.), Temporal Redundancy (Checkpoints and Backward Recovery, etc.)

- ❑ Software errors become dominant as system's complexity increases
 - ❑ (e.g.) Several bugs per kilo lines
- ❑ Hard to debug, and redundancy techniques are expensive
 - ❑ (e.g.) Backward recovery with checkpoints is inappropriate for real-time applications

Network Errors and Error Control Schemes

Failures	Causes	Metrics	Traditional Approaches
Data Losses, Deadline Misses, Node (Link) Failure, System Down	Network Congestion, Noise/Interference, Malicious Attacks	Packet Loss Rate, Deadline Miss Rate, SNR, MTTF, MTBF, MTTR	Resource Reservation, Data Redundancy (CRC, etc.), Temporal Redundancy (Retransmission, etc.), Spatial Redundancy (Replicated Nodes, MIMO, etc.)

- ❑ Omission Errors – lost/dropped messages
- ❑ Network is unreliable (especially, wireless networks)

- SNR: Signal to Noise Ratio
- MTTR: Mean Time To Recovery
- CRC: Cyclic Redundancy Check
- MIMO: Multiple-In Multiple-Out

■ Buffer overflow, Collisions at the MAC layer, Receiver out of range

- ❑ Joint approaches across OSI layers have been investigated for

Classifying fault-tolerance

Fail-safe tolerance

Given safety predicate is preserved, but liveness may be affected

Example. Due to failure, no process can enter its critical section for an indefinite period. In a traffic crossing, failure changes the traffic in both directions to red.

Graceful degradation

Application continues, but in a “degraded” mode. Much depends on what kind of degradation is acceptable.

Example. Consider message-based mutual exclusion. Processes will enter their critical sections, but not in timestamp order.

Conventional Approaches

- Build redundancy into hardware/software
 - Modular [Redundancy](#), [N-Version Programming](#) Conventional TRM (Triple Modular Redundancy) can incur 200% overheads without optimization.
 - Replication of tasks and processes may result in overprovisioning
 - Error Control Coding
- Checkpointing and rollbacks
 - Usually accomplished through logging (e.g. messages)
 - Backward Recovery with Checkpoints cannot guarantee the completion time of a task.
- Hybrid
 - Recovery Blocks

Defining Consensus

- N processes
- Each process p has
 - input variable x_p : initially either 0 or 1
 - 1. output variable y_p : initially b (b =undecided) – can be changed only once
- **Consensus problem:** design a protocol so that either
 1. all non-faulty processes set their output variables to 0
 2. Or non-faulty all processes set their output variables to 1
 3. There is at least one initial state that leads to each outcomes 1 and 2 above

Solving Consensus

- No failures – trivial
 - All-to-all broadcast
- With failures
 - Assumption: Processes fail only by *crash-stopping*
- Synchronous system: bounds on
 - Message delays
 - Max time for each process step
 - e.g., multiprocessor (common clock across processors)
- Asynchronous system: no such bounds!
 - e.g., The Internet! The Web!

Asynchronous Consensus

- Messages have arbitrary delay, processes arbitrarily slow
- **Impossible to achieve!**
 - a slow process indistinguishable from a crashed process
- **Theorem:** In a purely asynchronous distributed system, the consensus problem is impossible to solve if even a single process crashes
- Result due to Fischer, Lynch, Patterson (commonly known as FLP 85).

Failure detection

The design of fault-tolerant algorithms will be simple if processes can detect failures.

- In synchronous systems with bounded delay channels, crash failures can **definitely be detected** using timeouts.
- In asynchronous distributed systems, the detection of **crash failures** is imperfect.
- **Completeness** – Every crashed process is suspected
- **Accuracy** – No correct process is suspected.

Classification of completeness

- **Strong completeness.** Every crashed process is eventually suspected by *every correct process*, and remains a suspect thereafter.
- **Weak completeness.** Every crashed process is eventually suspected by *at least one* correct process, and remains a suspect thereafter.
- **Strong accuracy.** No correct process is ever suspected.
- **Weak accuracy.** There is at least one correct process that is never suspected.

Note that we don't care what mechanism is used for suspecting a process.

Classifying failure detectors

Perfect P. (Strongly) Complete and strongly accurate

Strong S. (Strongly) Complete and weakly accurate

Eventually perfect $\diamond P$.

(Strongly) Complete and eventually strongly accurate

Eventually strong $\diamond S$

(Strongly) Complete and eventually weakly accurate

Other classes are feasible: W (weak completeness) and weak accuracy) and $\diamond W$

Replication

❖ Enhance a service by replicating data

❖ Increased Availability

- ❖ Of service. When servers fail or when the network is partitioned.

❖ Fault Tolerance

- ❖ Under the fail-stop model, if up to f of $f+1$ servers crash, at least one is alive.

❖ Load Balancing

- ❖ One approach: Multiple server IPs can be assigned to the same name in DNS, which returns answers round-robin.

P : probability that one server fails = $1 - P$ = availability of service. e.g. $P = 5\% \Rightarrow$ service is available 95% of the time.

P^n : probability that n servers fail = $1 - P^n$ = availability of service. e.g. $P = 5\%$, $n = 3 \Rightarrow$ service available 99.875% of the time

Replication Management

❖ Request Communication

- ❖ Requests can be made to a single RM or to multiple RMs

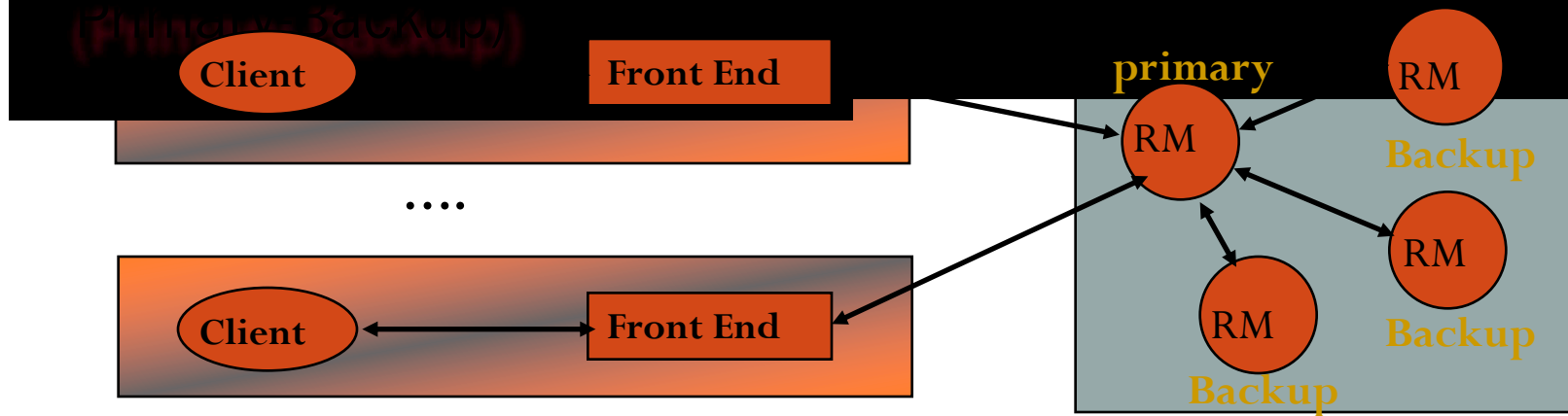
❖ Coordination: The RMs decide

- ❖ whether the request is to be applied
- ❖ the order of requests
 - ❖ **FIFO ordering**: If a FE issues r then r' , then any correct RM handles r and then r' .
 - ❖ **Causal ordering**: If the issue of r “happened before” the issue of r' , then any correct RM handles r and then r' .
 - ❖ **Total ordering**: If a correct RM handles r and then r' , then any correct RM handles r and then r' .

❖ Execution: The RMs execute the request (often they do this tentatively).

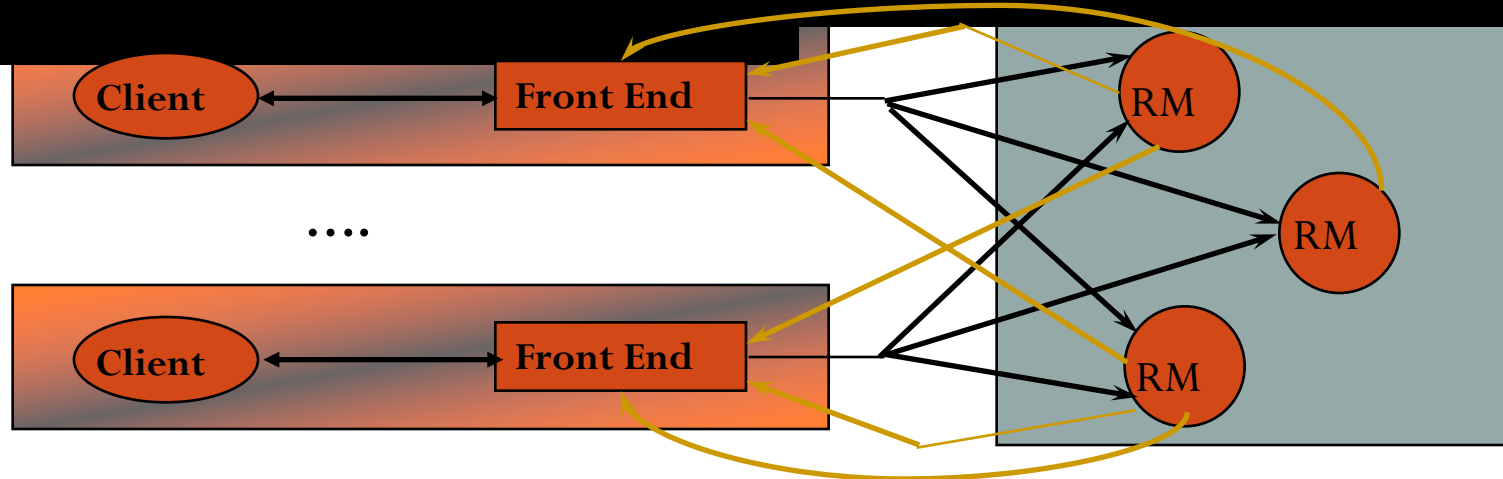
Passive Replication

(Primary-Backup)



- ❖ **Request Communication:** the request is issued to the primary RM and carries a unique request id.
- ❖ **Coordination:** Primary takes requests atomically, in order, checks id (resends response if not new id.)
- ❖ **Execution:** Primary executes & stores the response
- ❖ **Agreement:** If update, primary sends updated state/result, req-id and response to all backup RMs (1-phase commit enough).
- ❖ **Response:** primary sends result to the front end

Active Replication



- ❖ **Request Communication:** The request contains a unique identifier and is multicast to all by a reliable totally-ordered multicast.
- ❖ **Coordination:** Group communication ensures that requests are delivered to each RM in the same order (but may be at different physical times!).
- ❖ **Execution:** Each replica executes the request. (Correct replicas return same result since they are running the same program, i.e., they are *replicated protocols* or *replicated state machines*)
- ❖ **Agreement:** **No** agreement phase is needed, because of multicast delivery semantics of requests
- ❖ **Response:** Each replica sends response directly to FE

Message Logging

- Tolerate crash failures
- Each process periodically records its local state and log messages received after
 - Once a crashed process recovers, its state must be consistent with the states of other processes
 - Orphan processes
 - surviving processes whose states are inconsistent with the recovered state of a crashed process
 - Message Logging protocols guarantee that upon recovery no processes are orphan processes
 - Pessimistic Logging – avoid creation of orphans
 - Optimistic Logging – eliminate orphans during recovery
 - Causal Logging -- no orphans when failures happen and do not block processes when failures do not occur (add info to messages)